

Lesson 11 Structured Note: Parenthesis Matching Using a Linked-List Char Stack

0. Current Position in the Course

You are now in **Lesson 11** of the stack curriculum.

Before Lesson 11, you learned several important ideas:

Lesson	Main idea
Lesson 1	A stack follows LIFO: Last In, First Out .
Lesson 2	An array stack uses <code>size</code> , <code>top</code> , and an array pointer.
Lesson 3	Array stack operations include <code>push</code> , <code>pop</code> , <code>peek</code> , <code>stackTop</code> , <code>isEmpty</code> , and <code>isFull</code> .
Lesson 4	A complete C array-stack program combines all array-stack functions.
Lesson 5	A linked-list stack uses nodes and a <code>top</code> pointer.
Lesson 6	Linked-list stack operations include <code>push</code> , <code>pop</code> , <code>peek</code> , <code>stackTop</code> , <code>isEmpty</code> , and <code>isFull</code> .
Lesson 7	A complete C linked-list stack uses <code>struct Node</code> , a global <code>top</code> pointer, <code>push</code> , <code>pop</code> , <code>display</code> , and <code>main</code> .
Lesson 8	A C++ class can be used to implement a linked-list stack more safely.
Lesson 9	Parenthesis matching was introduced as a stack application.
Lesson 10	Parenthesis matching was implemented using an array char stack .

Lesson 11 continues parenthesis matching, but now we use a **linked-list char stack** instead of an array char stack.

The logic is the same as Lesson 10:

```
If you see '(' -> push it
If you see ')' -> pop one '('
Ignore everything else
```

The main change is the stack implementation.

In Lesson 10:

```
Array stack empty condition: top == -1
```

In Lesson 11:

```
Linked-list stack empty condition: top == NULL
```

1. Lesson 11 Goal

By the end of this lesson, you should understand:

- What parenthesis matching means.
- Why stacks are useful for parenthesis matching.
- Why Lesson 11 uses a linked-list stack.
- Why the node stores `char` instead of `int`.
- How to define a linked-list node for characters.
- What the global `top` pointer means.
- Why `top == NULL` means the stack is empty.
- How `push(char x)` works.
- How `pop()` works.
- How to scan an expression character by character.
- Why opening parentheses are pushed.
- Why closing parentheses cause a pop.
- Why we must check `top == NULL` before popping.
- How to detect an extra opening parenthesis.
- How to detect an extra closing parenthesis.
- Why non-parenthesis characters are ignored.
- Why the final condition is `top == NULL`.
- How to write the full C program.
- How to dry-run balanced and unbalanced expressions.
- The time and space complexity of the solution.
- Common beginner mistakes.

2. Quick Review: What Problem Are We Solving?

We want to check whether parentheses in an expression are **balanced**.

Balanced means:

1. Every opening parenthesis `(` has a matching closing parenthesis `)`.
2. Every closing parenthesis `)` has a previous unmatched opening parenthesis `(`.
3. Parentheses close in the correct order.
4. No opening parenthesis is left at the end.
5. No closing parenthesis appears too early.

Example 1: Balanced

`(A+B)`

This is balanced because the opening `(` is matched by the closing `)`.

Example 2: Not Balanced Because of Extra Opening Parenthesis

`((A+B)`

There are two opening parentheses but only one closing parenthesis.

One `(` is left over.

So the expression is **not balanced**.

Example 3: Not Balanced Because of Extra Closing Parenthesis

`(A+B))`

The first `)` matches the opening `(`.

The second `)` has no matching opening parenthesis.

So the expression is **not balanced**.

3. Why a Stack Is Useful for Parenthesis Matching

A stack follows:

LIFO = Last In, First Out

This means:

The last item inserted is the first item removed.

Parentheses also behave this way.

Look at this expression:

((A+B))

The second (is opened after the first (.

So the second (must close before the first (closes.

That is exactly LIFO behavior:

Last opened parenthesis = first parenthesis that must be closed

That is why a stack is a good tool for parenthesis matching.

4. Main Rule of the Algorithm

When scanning the expression from left to right:

Character found	Action
(	Push it onto the stack.
)	Pop one opening parenthesis from the stack.
Anything else	Ignore it.

Simple meaning:

Opening parenthesis '(' means: I am waiting for a matching ')'.
Closing parenthesis ')' means: I should match the most recent unmatched '('.

Example:

$(A+B)*(C-D)$

The algorithm only cares about:

$() ()$

It ignores:

$A + B * C - D$

5. What Changes from Lesson 10?

Lesson 10 used an array character stack.

Lesson 11 uses a linked-list character stack.

Lesson 10 Array Char Stack

```
struct Stack {  
    int size;  
    int top;  
    char *S;  
};
```

In the array version:

`top == -1`

means the stack is empty.

Lesson 11 Linked-List Char Stack

```
struct Node {  
    char data;  
    struct Node *next;  
};
```

```
struct Node *top = NULL;
```

In the linked-list version:

```
top == NULL
```

means the stack is empty.

Key Difference

Concept	Array char stack	Linked-list char stack
Storage	Character array	Linked nodes
Meaning of <code>top</code>	Index	Pointer
Empty condition	<code>top == -1</code>	<code>top == NULL</code>
Size	Fixed after allocation	Grows as nodes are created
Push memory	Uses existing array slot	Allocates a new node
Pop memory	Decreases <code>top</code>	Frees removed node

6. Why the Node Stores `char`

Earlier linked-list stacks stored integers:

```
struct Node {  
    int data;  
    struct Node *next;  
};
```

That was useful for values like:

```
10, 20, 30
```

But parenthesis matching stores characters like:

```
'('
')
```

So the node data type must change from `int` to `char`.

For Lesson 11, the node becomes:

```
struct Node {
    char data;
    struct Node *next;
};
```

Important change:

```
int data;    // old integer stack
char data;   // new parenthesis matching stack
```

7. Linked-List Stack Setup

A linked-list stack is made of nodes.

Each node has two parts:

Field	Meaning
<code>data</code>	Stores the actual character, such as <code>'('</code> .
<code>next</code>	Points to the next node.

Code:

```
struct Node {
    char data;
    struct Node *next;
};
```

Then we create a global `top` pointer:

```
struct Node *top = NULL;
```

At the beginning:

```
top -> NULL
```

This means the stack is empty.

8. Meaning of `top` in a Linked-List Stack

In an array stack, `top` is an index.

Example:

```
top = 2;
```

That means the top value is stored at array index 2.

But in a linked-list stack, `top` is a pointer.

Example:

```
top -> '(' -> '(' -> NULL
```

Here:

- `top` points to the first node.
- The first node is the top of the stack.
- The rest of the nodes are below the top.

The empty condition is:

```
top == NULL
```

This means:

There is no first node.
The stack is empty.

9. Why We Use the Front of the Linked List as the Top

In a linked-list stack, the front node is used as the top.

Example:

```
top -> '(' -> '(' -> NULL
```

If we push a new `(`, we add it at the front:

```
top -> '(' -> '(' -> '(' -> NULL
```

Why the front?

Because `top` already points to the front node.

So pushing and popping at the front are fast:

```
O(1)
```

If we tried to push or pop at the end, we would need to scan the whole list.

That would take:

```
O(n)
```

A stack should work at the top efficiently, so the front node is the best choice.

10. Empty and Full Conditions in Lesson 11

Empty Condition

A linked-list stack is empty when:

```
top == NULL
```

Example:

```
top -> NULL
```

There is no node in the stack.

Full Condition

A linked-list stack does not have a fixed size like an array stack.

It grows by creating new nodes using `malloc()`.

So the stack is full only if heap memory is not available.

That happens when:

```
malloc() returns NULL
```

In `push()`, we check:

```
if (t == NULL) {  
    printf("Stack Overflow\n");  
}
```

Simple meaning:

```
Linked-list overflow means memory allocation failed.
```

11. The `push(char x)` Operation

Meaning

`push(char x)` inserts a character at the top of the stack.

For parenthesis matching, we mainly push:

'('

Example before push:

top -> NULL

Call:

`push('(');`

After push:

top -> '(' -> NULL

12. Code for `push(char x)`

```
void push(char x) {
    struct Node *t;
    t = (struct Node *)malloc(sizeof(struct Node));

    if (t == NULL) {
        printf("Stack Overflow\n");
    } else {
        t->data = x;
        t->next = top;
        top = t;
    }
}
```

13. Explanation of `push(char x)` Step by Step

Step 1: Create a temporary pointer

```
struct Node *t;
```

`t` will point to the new node.

Step 2: Allocate memory for one node

```
t = (struct Node *)malloc(sizeof(struct Node));
```

This asks the heap for enough memory to create one new node.

Step 3: Check whether allocation failed

```
if (t == NULL) {  
    printf("Stack Overflow\n");  
}
```

If `malloc()` returns `NULL`, the node was not created.

That means stack overflow.

Step 4: Store the character

```
t->data = x;
```

If we call:

```
push('(');
```

then:

```
t->data = '(';
```

Step 5: Link the new node to the old top

```
t->next = top;
```

The new node points to the previous top node.

Step 6: Move `top` to the new node

```
top = t;
```

Now the new node becomes the top of the stack.

14. The Key Push Pattern

The most important push pattern is:

```
t->next = top;  
top = t;
```

Simple meaning:

```
The new node points to the old top.  
Then top moves to the new node.
```

The order matters.

Correct Order

```
t->next = top;  
top = t;
```

Wrong Order

```
top = t;  
t->next = top;
```

The wrong order can damage the stack.

It may make the new node point to itself or lose the old stack.

15. `push()` Dry Run

Start:

```
top -> NULL
```

Call:

```
push('(');
```

Step 1

Create a new node:

```
t -> '('
```

Step 2

Set:

```
t->next = top;
```

Since `top` is currently `NULL`, the new node points to `NULL`:

```
t -> '(' -> NULL
```

Step 3

Set:

```
top = t;
```

Now:

```
top -> '(' -> NULL
```

The stack now contains one opening parenthesis.

16. The `pop()` Operation

Meaning

`pop()` removes and returns the top character from the stack.

For parenthesis matching, we pop when we see:

```
' ) '
```

Example before pop:

```
top -> '(' -> NULL
```

Call:

```
pop();
```

After pop:

```
top -> NULL
```

Returned character:

```
' ('
```

17. Code for `pop()`

```
char pop() {  
    struct Node *p;
```

```
char x = '\0';

if (top == NULL) {
    printf("Stack Underflow\n");
} else {
    p = top;
    x = p->data;
    top = top->next;
    free(p);
}

return x;
}
```

18. Explanation of pop() Step by Step

Step 1: Create a temporary pointer

```
struct Node *p;
```

`p` will point to the node that will be deleted.

Step 2: Create a return variable

```
char x = '\0';
```

`x` stores the character that will be returned.

If the stack is empty, `x` stays as `\0`.

`\0` is the null character.

It means "no useful character returned" in this simple beginner code.

Step 3: Check underflow

```
if (top == NULL) {
    printf("Stack Underflow\n");
}
```


If `top == NULL`, the stack is empty.

There is no node to remove.

This is called stack underflow.

Step 4: Save the old top node

```
p = top;
```

Now `p` points to the node that will be removed.

Step 5: Save the data

```
x = p->data;
```

This saves the character before deleting the node.

Step 6: Move `top` to the next node

```
top = top->next;
```

Now the second node becomes the new top.

If there was only one node, `top` becomes `NULL`.

Step 7: Free the old top node

```
free(p);
```

This releases the heap memory used by the removed node.

Step 8: Return the saved character

```
return x;
```

19. The Key Pop Pattern

The most important pop pattern is:

```
p = top;  
x = p->data;  
top = top->next;  
free(p);  
return x;
```

Simple meaning:

```
Save the old top node.  
Save its character.  
Move top to the next node.  
Free the old top node.  
Return the saved character.
```

The order matters.

Correct Idea

```
p = top;  
top = top->next;  
free(p);
```

Wrong Idea

```
free(top);  
top = top->next;
```

The wrong idea is unsafe.

After freeing `top`, using `top->next` is invalid.

20. `pop()` Dry Run

Start:

```
top -> '(' -> NULL
```

Call:

```
pop();
```

Step 1

Check:

```
top == NULL
```

This is false because `top` points to a node.

Step 2

Save old top:

```
p = top;
```

Now:

```
p -> '('
```

Step 3

Save data:

```
x = p->data;
```

So:

```
x = '(';
```

Step 4

Move top:

```
top = top->next;
```

Now:

```
top -> NULL
```

Step 5

Free old node:

```
free(p);
```

The node containing (is deleted from heap memory.

Step 6

Return:

```
'('
```

Final stack:

```
top -> NULL
```

21. Why We Check Empty Before Popping in

isBalanced()

When the algorithm sees), it wants to pop a previous (.

But what if the stack is already empty?

Example:

```
A+B)
```

There is a closing parenthesis, but there was no earlier opening parenthesis.

So this is not balanced.

That is why we check:

```
if (top == NULL) {  
    return 0;  
}
```

before calling:

```
pop();
```

This prevents popping from an empty stack.

It also detects an extra closing parenthesis.

22. The `isBalanced()` Function

The main function is:

```
int isBalanced(char *exp)
```

It receives an expression as a character array.

It returns:

Return value	Meaning
1	Parentheses are balanced.
0	Parentheses are not balanced.

23. `isBalanced()` Code

```
int isBalanced(char *exp) {  
    for (int i = 0; exp[i] != '\0'; i++) {
```

```

        if (exp[i] == '(') {
            push(exp[i]);
        }
        else if (exp[i] == ')') {
            if (top == NULL) {
                return 0;
            }
            pop();
        }
    }

    return top == NULL;
}

```

24. Explanation of `isBalanced()` Step by Step

Step 1: Scan the expression

```
for (int i = 0; exp[i] != '\0'; i++)
```

This scans the expression from left to right.

It stops when it reaches the null terminator `\0`.

Step 2: If the character is `(`, push it

```

if (exp[i] == '(') {
    push(exp[i]);
}

```

Simple meaning:

This opening parenthesis is waiting to be matched later.

Step 3: If the character is `)`, check the stack

```

else if (exp[i] == ')') {
    if (top == NULL) {
        return 0;
    }
}

```

```
}  
pop();  
}
```

If `top == NULL`, there is no previous `(` available.

So the expression is not balanced.

If the stack is not empty, we pop one `(`.

Simple meaning:

This closing parenthesis matched one previous opening parenthesis.

Step 4: Ignore all other characters

Characters like these are ignored:

A, B, C, D, numbers, +, -, *, /, spaces

Only `(` and `)` affect the stack.

Step 5: Final check

```
return top == NULL;
```

At the end, the stack must be empty.

If `top == NULL`, every opening parenthesis was matched.

If `top != NULL`, some opening parenthesis was never closed.

25. Important Note About `\0`

In C, a string ends with a special character:

```
'\0'
```

This is called the null terminator.

Example:

```
"(A+B)"
```

is stored like this:

```
(' ' 'A' '+' 'B' ' ') '\0'
```

The loop:

```
for (int i = 0; exp[i] != '\0'; i++)
```

means:

```
Keep scanning until the string ends.
```

The null terminator is not pushed.

It is only used to stop the loop.

26. Complete Beginner C Program

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    char data;
    struct Node *next;
};

struct Node *top = NULL;

void push(char x) {
    struct Node *t;
    t = (struct Node *)malloc(sizeof(struct Node));

    if (t == NULL) {
```



```

        printf("Stack Overflow\n");
    } else {
        t->data = x;
        t->next = top;
        top = t;
    }
}

char pop() {
    struct Node *p;
    char x = '\0';

    if (top == NULL) {
        printf("Stack Underflow\n");
    } else {
        p = top;
        x = p->data;
        top = top->next;
        free(p);
    }

    return x;
}

void clearStack() {
    struct Node *p;

    while (top != NULL) {
        p = top;
        top = top->next;
        free(p);
    }
}

int isBalanced(char *exp) {
    clearStack();

    for (int i = 0; exp[i] != '\0'; i++) {
        if (exp[i] == '(') {
            push(exp[i]);
        }
        else if (exp[i] == ')') {
            if (top == NULL) {
                clearStack();
                return 0;
            }
            pop();
        }
    }
}

```

```

    }

    int result = (top == NULL);
    clearStack();

    return result;
}

int main() {
    char exp[] = "(A+B)*(C-D)";

    if (isBalanced(exp)) {
        printf("Balanced\n");
    } else {
        printf("Not Balanced\n");
    }

    return 0;
}

```

27. Why This Program Includes `clearStack()`

The simplest Lesson 11 version may only use:

```

push();
pop();
isBalanced();

```

But the beginner-friendly complete program above also includes:

```

void clearStack()

```

This function removes any remaining nodes from the stack.

Why is this useful?

1. It prevents memory leaks.
2. It resets the global stack before checking a new expression.
3. It cleans leftover opening parentheses after an unbalanced expression.

Code:

```
void clearStack() {  
    struct Node *p;  
  
    while (top != NULL) {  
        p = top;  
        top = top->next;  
        free(p);  
    }  
}
```

Simple meaning:

While the stack is not empty, delete the top node.

28. Expected Output

For this expression:

$(A+B)*(C-D)$

The program prints:

Balanced

29. Dry Run 1: Balanced Expression

Expression:

$(A+B)*(C-D)$

Only parentheses matter:

$(\) (\)$

Initial stack:

top -> NULL

Trace:

Step	Character	Action	Stack after action
1	(	push	top -> (
2	A+B	ignore	top -> (
3	)	pop	top -> NULL
4	*	ignore	top -> NULL
5	(	push	top -> (
6	C-D	ignore	top -> (
7	)	pop	top -> NULL

At the end:

top == NULL

So the result is:

Balanced

Reason:

Every opening parenthesis got a matching closing parenthesis.

30. Dry Run 2: Extra Opening Parenthesis

Expression:

((A+B)

Only parentheses matter:

(()

Initial stack:

top -> NULL

Trace:

Step	Character	Action	Stack after action
1	(	push	top -> (
2	(	push	top -> (-> (
3	A+B	ignore	top -> (-> (
4	)	pop	top -> (

At the end:

top != NULL

One opening parenthesis is still waiting to be closed.

Result:

Not Balanced

Reason:

There is an extra opening parenthesis.

This error is detected at the final check:

```
return top == NULL;
```

31. Dry Run 3: Extra Closing Parenthesis

Expression:

(A+B))

Only parentheses matter:

())

Initial stack:

top -> NULL

Trace:

Step	Character	Action	Stack after action
1	(	push	top -> (
2	A+B	ignore	top -> (
3	)	pop	top -> NULL
4	)	stack empty, cannot pop	error

At the last): :

top == NULL

There is no matching opening parenthesis.

Result:

Not Balanced

Reason:

There is an extra closing parenthesis.

This error is detected immediately while scanning.

32. Dry Run 4: Expression with No Parentheses

Expression:

A+B*C

The algorithm scans each character:

Character	Action
A	ignore
+	ignore
B	ignore
*	ignore
C	ignore

No parentheses are pushed.

No parentheses are popped.

At the end:

top == NULL

Result:

Balanced

Why?

Because there are no unmatched parentheses.

33. Dry Run 5: Balanced but Mathematically Invalid Expression

Expression:

(A+)*(B)

Parentheses only:

() ()

The parenthesis matching algorithm says:

Balanced

But the mathematical expression is invalid because:

A+

is missing an operand after `+`.

Important lesson:

Parenthesis matching checks only parentheses.
It does not check full mathematical correctness.

34. Two Main Error Types

Error Type 1: Extra Opening Parenthesis

Example:

((A+B)

What happens?

- Two `(` are pushed.
- Only one `)` appears.
- One `(` remains in the stack.

Detected when?

At the end of the scan.

Condition:

`top != NULL`

Result:

Not Balanced

Error Type 2: Extra Closing Parenthesis

Example:

`(A+B))`

What happens?

- The first `)` matches the `(`.
- The second `)` appears when the stack is empty.

Detected when?

Immediately while scanning.

Condition:

`if (top == NULL)`

before popping.

Result:

Not Balanced

35. What the Algorithm Checks

This algorithm checks:

Are the round parentheses balanced?

It checks whether:

- Every `(` has a matching `)`.
- No `)` appears without a previous unmatched `(`.
- No `(` remains at the end.
- Parentheses close in proper LIFO order.

36. What the Algorithm Does Not Check

This algorithm does not check whether the full mathematical expression is valid.

Example:

`(A + ) * ( - D)`

The parentheses are:

`( ) ( )`

So the algorithm says:

Balanced

But the expression is mathematically invalid because operands are missing.

So do not confuse these two questions:

Question	Answered by this algorithm?
Are parentheses balanced?	Yes
Is the full expression mathematically valid?	No

37. Time Complexity

Let:

n = length of the expression

The algorithm scans every character once.

So the time complexity is:

$O(n)$

Why?

Because the loop goes from the first character to the null terminator:

```
for (int i = 0; exp[i] != '\0'; i++)
```

Each character is processed once.

38. Space Complexity

The stack stores opening parentheses.

Worst case:

(((((

Every character is an opening parenthesis.

So the stack may store many nodes.

Worst-case space complexity:

$O(n)$

Why?

Because in the worst case, the number of pushed nodes grows with the expression length.

39. Cost of Individual Operations

Operation	Time complexity	Reason
<code>push()</code>	<code>O(1)</code>	Adds one node at the front.
<code>pop()</code>	<code>O(1)</code>	Removes the top node directly.
Empty check	<code>O(1)</code>	Checks whether <code>top == NULL</code> .
<code>isBalanced()</code>	<code>O(n)</code>	Scans the whole expression once.

40. Memory Management

A linked-list stack uses heap memory.

Every successful push creates one node:

```
t = (struct Node *)malloc(sizeof(struct Node));
```

Every successful pop deletes one node:

```
free(p);
```

Important rule:

```
Every node created with malloc() should eventually be released with free().
```

If we do not free removed or leftover nodes, the program wastes memory.

That problem is called a memory leak.

41. Array Char Stack vs Linked-List Char Stack

Feature	Array char stack	Linked-list char stack
Storage	Character array	Nodes
Empty condition	<code>top == -1</code>	<code>top == NULL</code>
Full condition	<code>top == size - 1</code>	<code>malloc()</code> returns <code>NULL</code>
Size	Fixed after allocation	Grows as needed until heap memory fails
Push	Store at <code>S[++top]</code>	Create new node and link at front
Pop	Return <code>S[top--]</code>	Delete top node and move <code>top</code>
Main advantage	Simple indexing	Flexible size
Main risk	Fixed capacity overflow	Memory leaks if <code>free()</code> is forgotten

42. Common Beginner Mistakes

Mistake 1: Using `top == -1` in a linked-list stack

Wrong:

```
if (top == -1)
```

Correct:

```
if (top == NULL)
```

Reason:

In a linked-list stack, `top` is a pointer, not an integer index.

Mistake 2: Forgetting to change `int data` to `char data`

Wrong for this lesson:

```
int data;
```

Correct:

```
char data;
```

Reason:

We are storing parentheses, and parentheses are characters.

Mistake 3: Forgetting to check before popping

Wrong idea:

```
pop();
```

immediately when `)` is found.

Correct idea:

```
if (top == NULL) {  
    return 0;  
}  
pop();
```

Reason:

A closing parenthesis may appear when there is no matching opening parenthesis.

Mistake 4: Forgetting the final check

Wrong idea:

If no error happened during scanning, the expression is balanced.

Correct idea:

```
return top == NULL;
```

Reason:

There may be leftover opening parentheses at the end.

Mistake 5: Moving `top` incorrectly in push

Wrong:

```
top = t;  
t->next = top;
```

Correct:

```
t->next = top;  
top = t;
```

Mistake 6: Freeing before moving `top`

Wrong:

```
free(top);  
top = top->next;
```

Correct:

```
p = top;  
top = top->next;  
free(p);
```

Mistake 7: Thinking balanced parentheses means valid math

Wrong idea:

If parentheses are balanced, the expression is mathematically valid.

Correct idea:

Balanced parentheses only means the parentheses match.
The expression may still be mathematically invalid.

Mistake 8: Forgetting to free leftover nodes

If the expression has extra opening parentheses, nodes may remain in the stack.

Example:

((A+B)

One (remains.

Use `clearStack()` to clean leftover nodes if needed.

43. Key Code Patterns to Memorize

Node structure

```
struct Node {  
    char data;  
    struct Node *next;  
};
```

Global top pointer

```
struct Node *top = NULL;
```

Empty check

```
top == NULL
```


Push pattern

```
t->data = x;  
t->next = top;  
top = t;
```

Pop pattern

```
p = top;  
x = p->data;  
top = top->next;  
free(p);  
return x;
```

Parenthesis matching pattern

```
if (exp[i] == '(') {  
    push(exp[i]);  
} else if (exp[i] == ')') {  
    if (top == NULL) {  
        return 0;  
    }  
    pop();  
}
```

Final check

```
return top == NULL;
```

44. Full Algorithm in Plain English

1. Start with an empty linked-list stack.
2. Scan the expression from left to right.
3. If the current character is `(`, push it.
4. If the current character is `)`, first check whether the stack is empty.
5. If the stack is empty, return not balanced.
6. If the stack is not empty, pop one `(`.
7. Ignore all other characters.
8. After scanning the whole expression, check the stack.

9. If the stack is empty, return balanced.
10. If the stack is not empty, return not balanced.

45. Pseudocode

```
isBalanced(expression):
    start with empty stack

    for each character ch in expression:
        if ch is '(':
            push ch onto stack

        else if ch is ')':
            if stack is empty:
                return false
            else:
                pop from stack

        else:
            ignore ch

    if stack is empty:
        return true
    else:
        return false
```

46. Mini Trace Practice

Trace this expression:

((A+B))

Only parentheses matter:

(())

Step	Character	Action	Stack after action
1	(	push	top -> (

Step	Character	Action	Stack after action
2	(	push	top -> (-> (
3	)	pop	top -> (
4	)	pop	top -> NULL

Final result:

Balanced

47. Checkpoint Questions

Try to answer these without looking above.

1. What does Lesson 11 use instead of an array char stack?
2. What data type should the node store?
3. What does `top == NULL` mean?
4. What does `push(' ( ')` do?
5. What should happen when the algorithm sees `)`?
6. Why must we check `top == NULL` before popping?
7. What does it mean if `top != NULL` at the end?
8. What does the algorithm ignore?
9. Does the algorithm check full mathematical correctness?
10. What is the time complexity of `isBalanced()`?
11. What is the worst-case space complexity?
12. Why must `free(p)` be used in `pop()`?
13. What is the main difference between Lesson 10 and Lesson 11?

48. Checkpoint Answers

1. Lesson 11 uses a linked-list char stack.
2. The node should store `char` data.
3. `top == NULL` means the linked-list stack is empty.
4. `push(' ( ')` creates a new node containing `(` and places it at the top.
5. When the algorithm sees `)`, it should check whether the stack is empty. If not empty, it should pop one `(`.
6. We check `top == NULL` before popping to detect an extra closing parenthesis and avoid stack underflow.
7. If `top != NULL` at the end, some opening parenthesis was never closed.
8. The algorithm ignores letters, numbers, operators, and spaces.

9. No. It only checks parenthesis balance, not full mathematical correctness.
 10. The time complexity is $O(n)$.
 11. The worst-case space complexity is $O(n)$.
 12. `free(p)` releases the heap memory of the removed node.
 13. Lesson 10 uses an array char stack. Lesson 11 uses a linked-list char stack.
-

49. Practice Tasks

Task 1

Trace this expression:

(A+B)

State whether it is balanced.

Task 2

Trace this expression:

((A+B)

Show which (remains in the stack.

Task 3

Trace this expression:

(A+B))

Show the exact step where the stack is empty but) appears.

Task 4

Explain why this is wrong in a linked-list stack:

top == -1

Task 5

Write the `struct Node` for Lesson 11 from memory.

Task 6

Write the key push pattern from memory:

```
t->next = top;
top = t;
```

Task 7

Write the key pop pattern from memory:

```
p = top;
top = top->next;
free(p);
```

Task 8

Explain why this expression may be balanced but invalid:

```
(A+)*(B)
```

50. Short Glossary

Term	Simple meaning
Stack	A data structure that follows LIFO.
LIFO	Last In, First Out.
Parenthesis matching	Checking whether every <code>(</code> has a matching <code>)</code> .
Linked-list stack	A stack implemented using connected nodes.
Node	One element of a linked list.
<code>data</code>	The value stored in a node. In Lesson 11, it is <code>char</code> .
<code>next</code>	Pointer to the next node.

Term	Simple meaning
<code>top</code>	Pointer to the top node of the stack.
<code>NULL</code>	Means a pointer points to nothing.
<code>top == NULL</code>	Empty linked-list stack condition.
<code>push(char x)</code>	Insert a character at the top.
<code>pop()</code>	Remove and return the top character.
<code>malloc()</code>	Allocates heap memory for a new node.
<code>free()</code>	Releases heap memory from a removed node.
Overflow	Trying to push when memory is unavailable.
Underflow	Trying to pop when the stack is empty.
Null terminator	The <code>\0</code> character that marks the end of a C string.
<code>O(1)</code>	Constant time.
<code>O(n)</code>	Time or space grows with input size.

51. Final Lesson 11 Summary

Lesson 11 teaches parenthesis matching using a linked-list char stack.

The main idea is simple:

Push every '('.
Pop for every ')'.
At the end, the stack must be empty.

The linked-list stack uses nodes:

```
struct Node {
    char data;
    struct Node *next;
};
```

The stack starts empty:

```
struct Node *top = NULL;
```

The empty condition is:

```
top == NULL
```

The most important changes from an integer linked-list stack are:

```
char data;  
void push(char x);  
char pop();
```

The most important parenthesis matching logic is:

```
if (exp[i] == '(') {  
    push(exp[i]);  
} else if (exp[i] == ')') {  
    if (top == NULL) {  
        return 0;  
    }  
    pop();  
}
```

At the end:

```
return top == NULL;
```

If `top == NULL`, the expression is balanced.

If `top != NULL`, there is at least one extra opening parenthesis.

If a `)` appears when `top == NULL`, there is an extra closing parenthesis.

Time complexity:

```
O(n)
```

Worst-case space complexity:

$O(n)$

Most important takeaway:

Lesson 11 keeps the same parenthesis matching idea from Lesson 10, but changes the storage from an array char stack to a linked-list char stack.